Testing

16: Simple Neural Networks

Some material taken from:
https://github.com/trekhleb/homemade-machine-learning/tree/master/homemade/neural_network
http://cs231n.github.io/neural-networks-1/

Engr 315: Hardware / Software Codesign

Andrew Lukefahr
Indiana University

- I don’t know how long / how bad.
- I don’t know the future.
- I’m OK with that.
- I’m happy to talk more privately if that’s helpful.

The Zoom / Canvas Lecture Plan

- Lecture using Zoom:
  - https://iu.zoom.us/j/362983973 ← website
- Also available on Canvas
  - Next day

Grading Plan

- IU added the option of “S”: Satisfactory
  - Let me know prior to Finals Week
- Replace final exam with final report.
- Report will be part of final project.

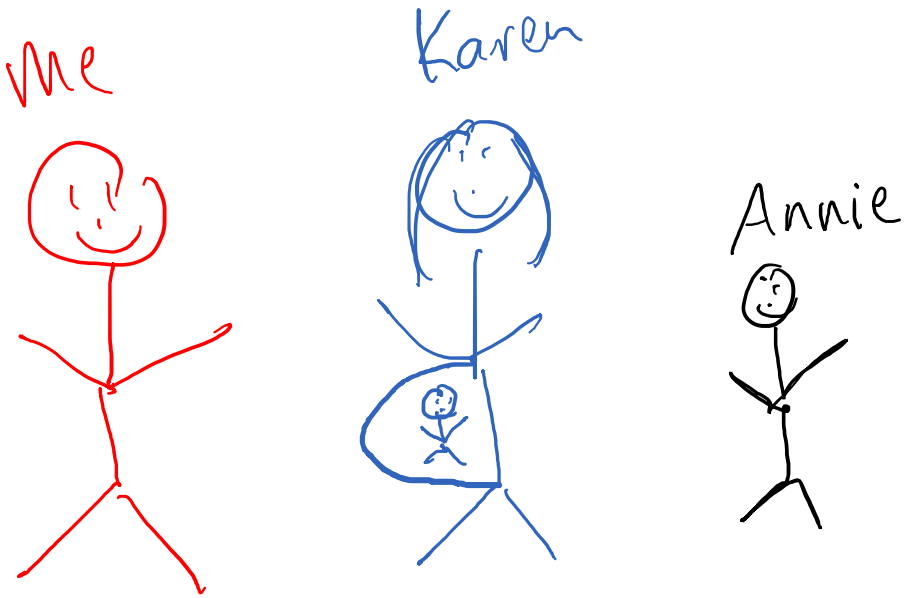
The Projects Plan

- Keep Project 4: Bitcounting
  - Due on Friday
- Skip Project 5: Floating Point
- Start on Project "X": Accelerate Machine Learning
  - Lecture on background today

Lecture Plan

- This week: Machine Learning
  - Simple Neural Networks
  - Python implementation
- Next weeks:
  - more performance evaluation tools
  - Bitcount in C.
  - Linux kernel modules
- If Time:
  - Intro to parallel processing

Lecture plan subject to change…

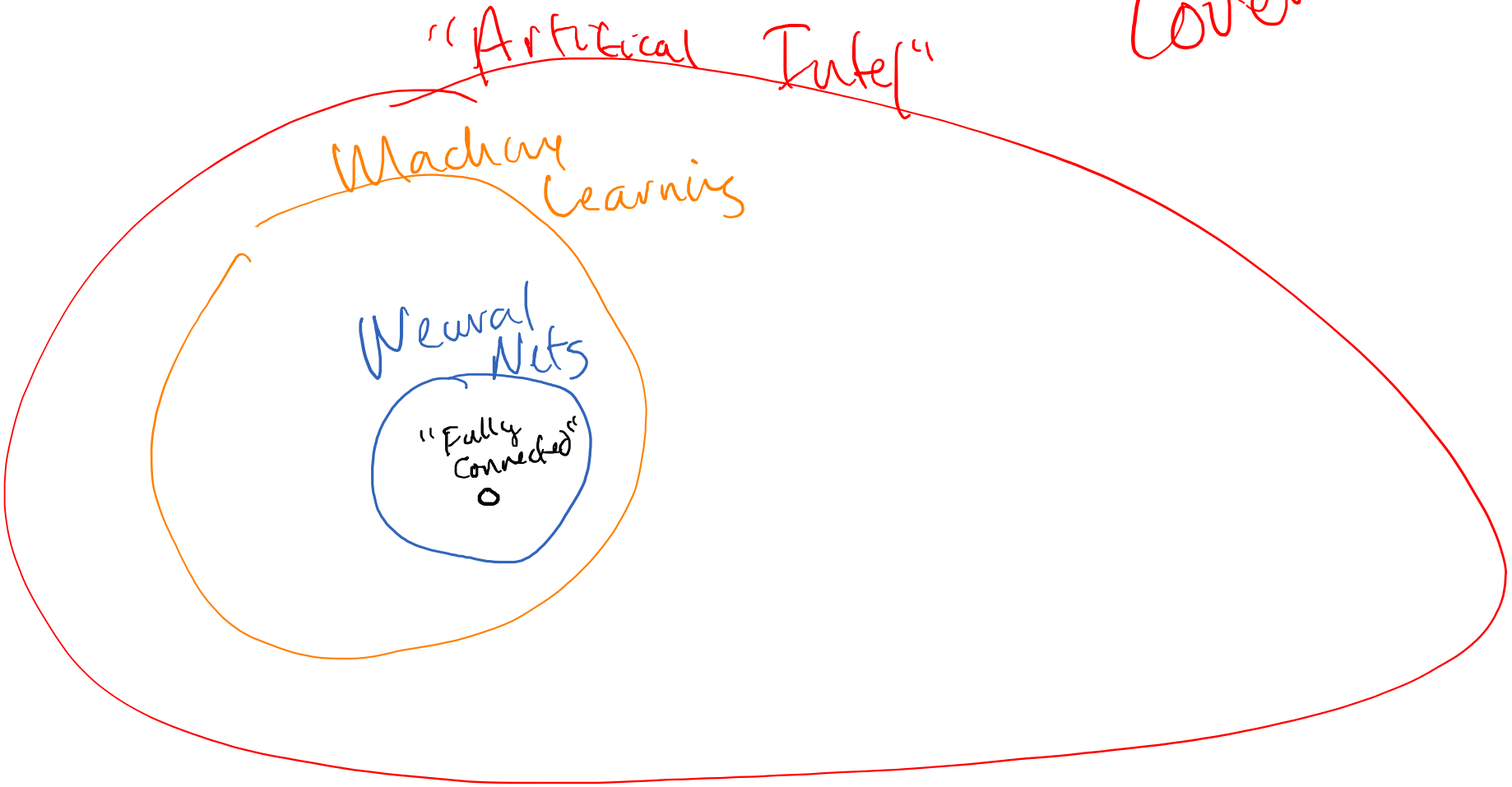


Project X: Accelerate Machine Learning

- Given: Python starter code to classify digits
- Your Task: Accelerate it!
- How: Up to you!
- Start on Python code today.
- Details on project later.

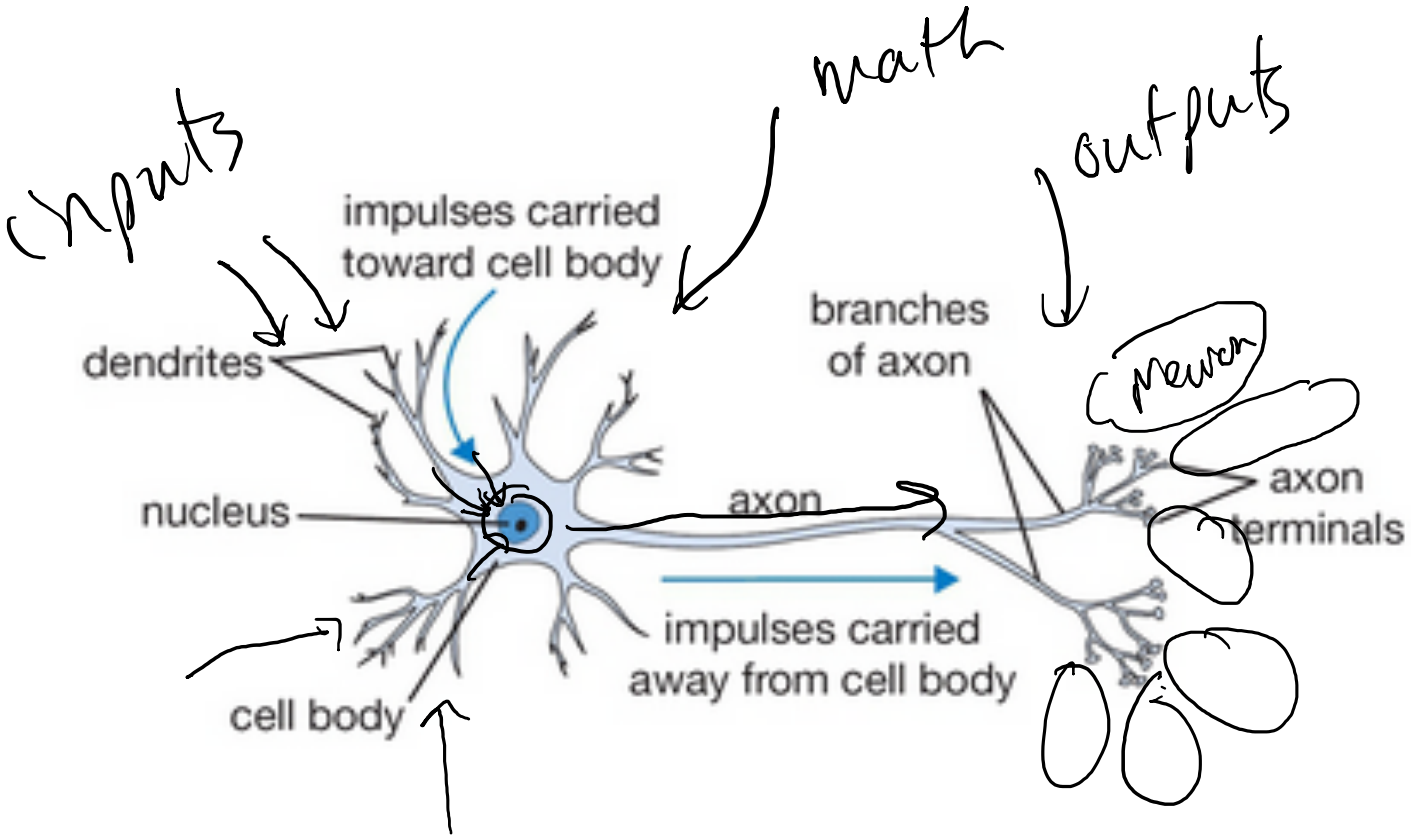
Simple Neural Networks

Covered in 221



The Biological Neuron

The Biological Neuron



The Math Neuron

Inputs X

$X_0 \times W_0$

$X_1 \times W_1$

$X_2 \times W_2$

Weight values

Add

$\sum x_i w_i$

Activation Function

$f(\sum x_i w_i)$

y

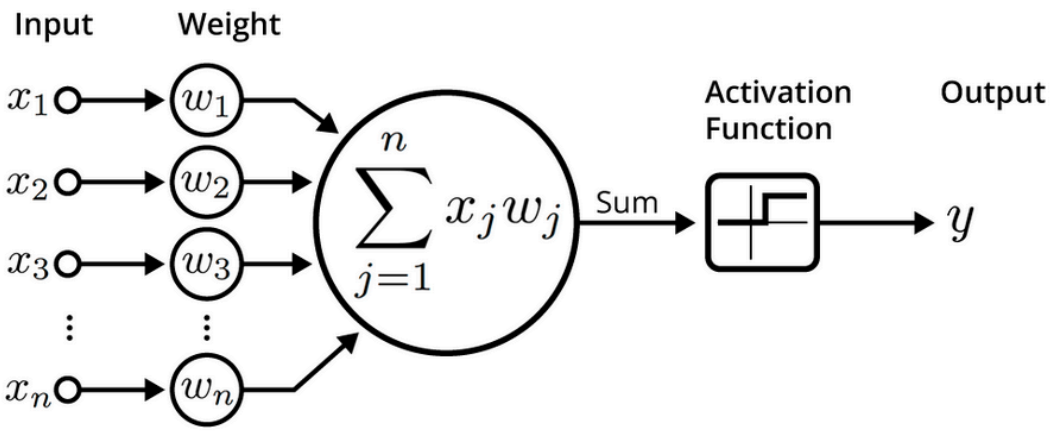
Tanh

relu → Rectified Linear

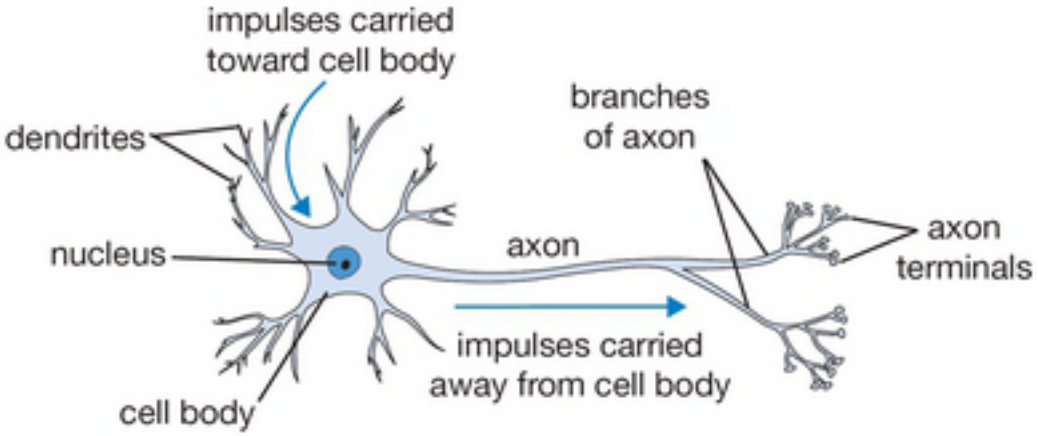
→ Sigmoid →

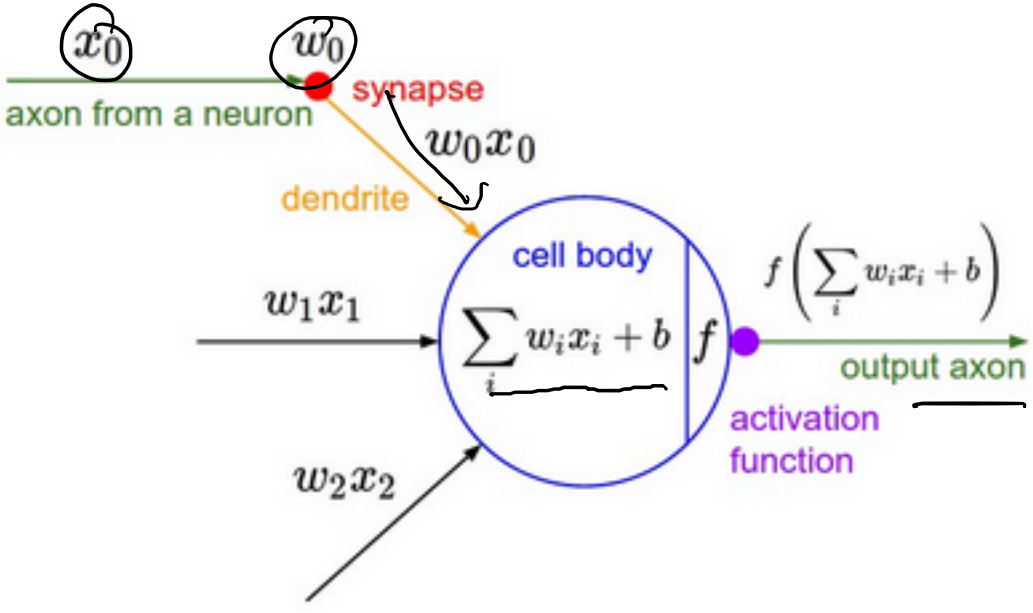
Log ???

The Math Neuron

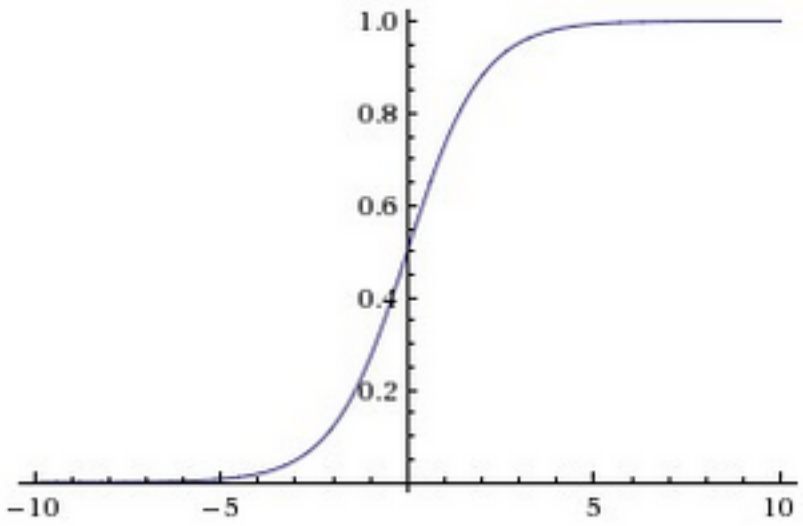


An illustration of an artificial neuron. Source: Becoming Human.





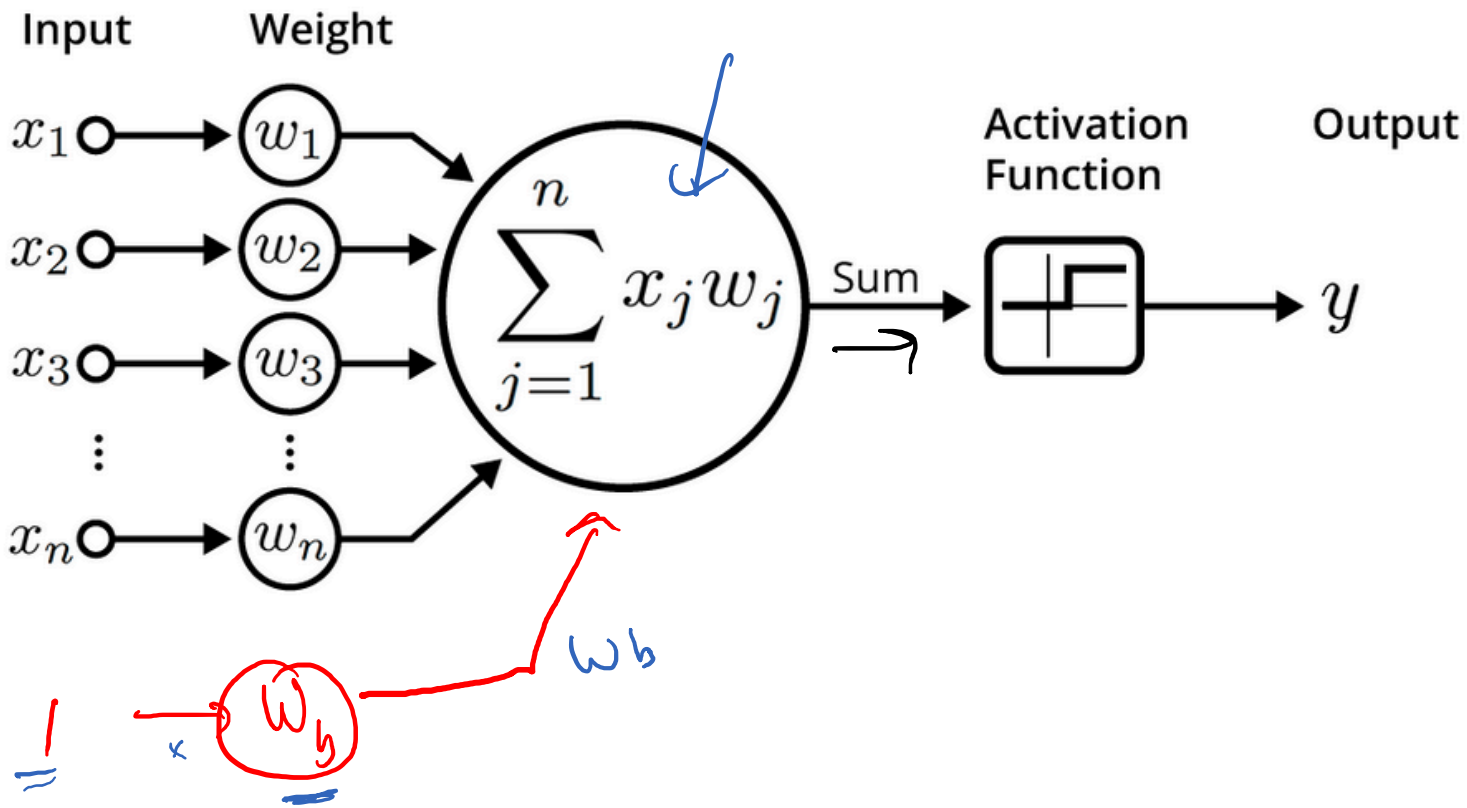
Activation Function: Sigmoid



$$\sigma(x)=1/(1+e^{-x})$$

Adding Bias

$$\sum_{j=1}^{n} x_j w_j + b \quad (\text{or } w_b)$$



Python Neuron

$$\sum \begin{matrix} x_0 \times w_0 \\ x_1 \times w_1 \\ x_2 \times w_2 \end{matrix} + \text{bias}$$

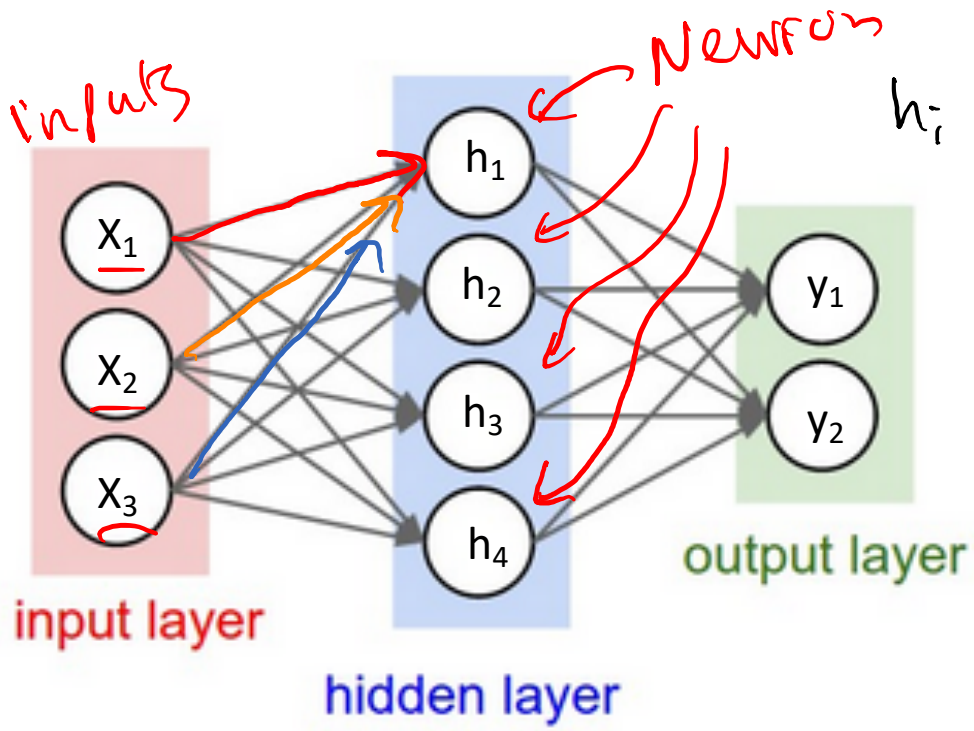
```
class Neuron(object):
  # ...
  def forward(self, inputs):
    """ assume inputs and weights are 1-D numpy arrays and bias is a number """
    cell_body_sum = np.sum(inputs * self.weights) + self.bias
    firing_rate = 1.0 / (1.0 + math.exp(-cell_body_sum)) # sigmoid activation function
    return firing_rate
```

sum

act. functn.

result

Multi-Neuron Networks



$h_1 = \text{sigmoid}(x_1 \cdot w_{11} + x_2 \cdot w_{12} + x_3 \cdot w_{13} + b)$

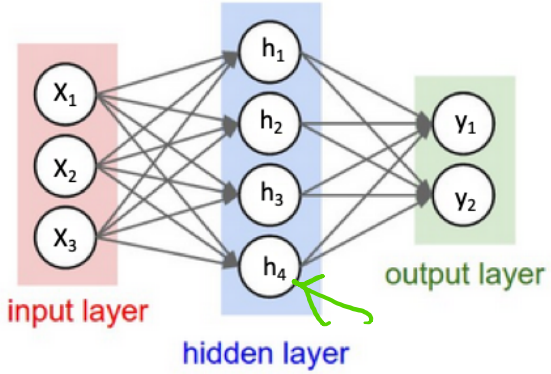
$h_2 = \text{sigmoid}(x_1 w_{21} + x_2 w_{22} + x_3 w_{23} + b)$

$h_3 = \text{sigmoid}(x_1 w_{31} + x_2 w_{32} + x_3 w_{33} + b)$

Input Layer: Just input values

Output Layers: No Activation Function

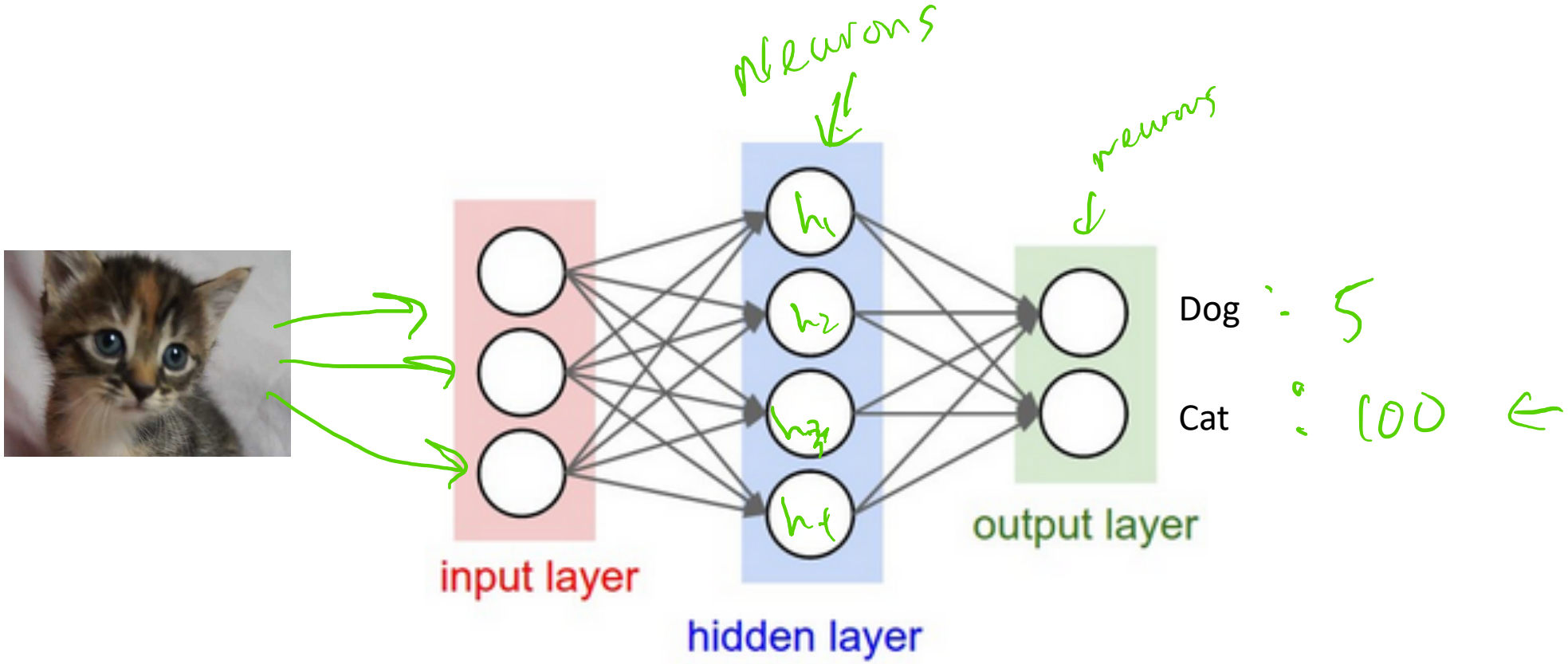
Vector Math



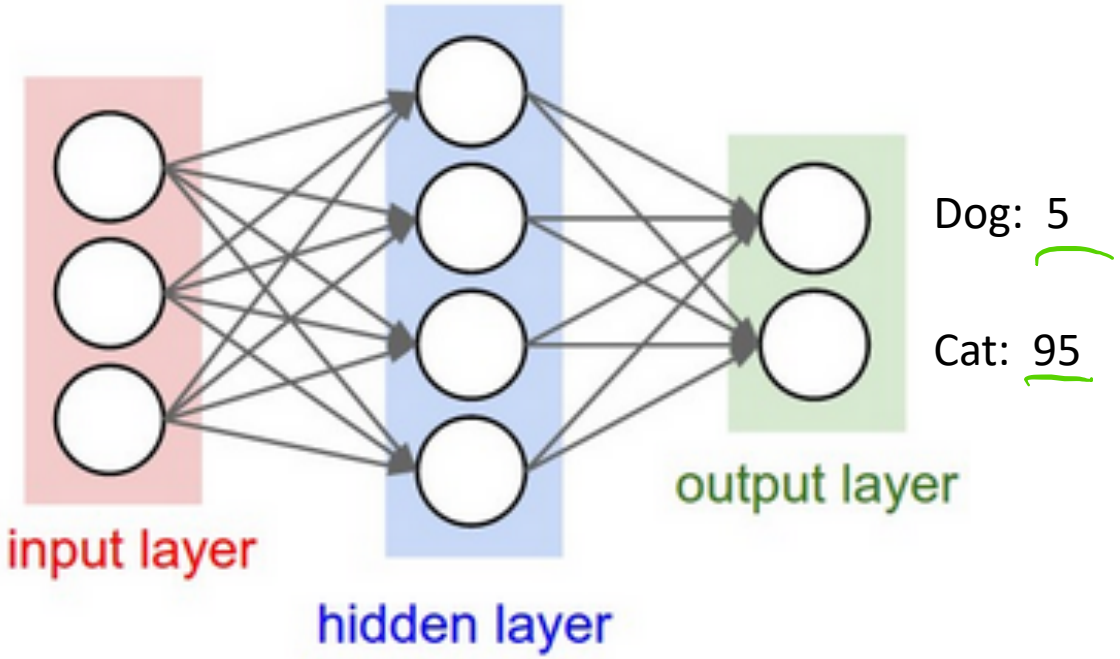
$$\begin{bmatrix} x_1 & x_2 & x_3 \end{bmatrix} \cdot \begin{bmatrix} w_{11} & w_{21} & w_{31} \\ w_{12} & w_{22} & w_{32} \\ w_{13} & w_{23} & w_{33} \end{bmatrix} = \begin{bmatrix} h_1, & h_2, & h_3, & h_4 \end{bmatrix}$$

$$\overbrace{x_1 \cdot w_{11} + x_2 \cdot w_{12} + x_3 \cdot w_{13}}^{h_1},\quad \overbrace{x_1 \cdot w_{21} + x_2 \cdot w_{22} + x_3 \cdot w_{23}}^{h_2},$$

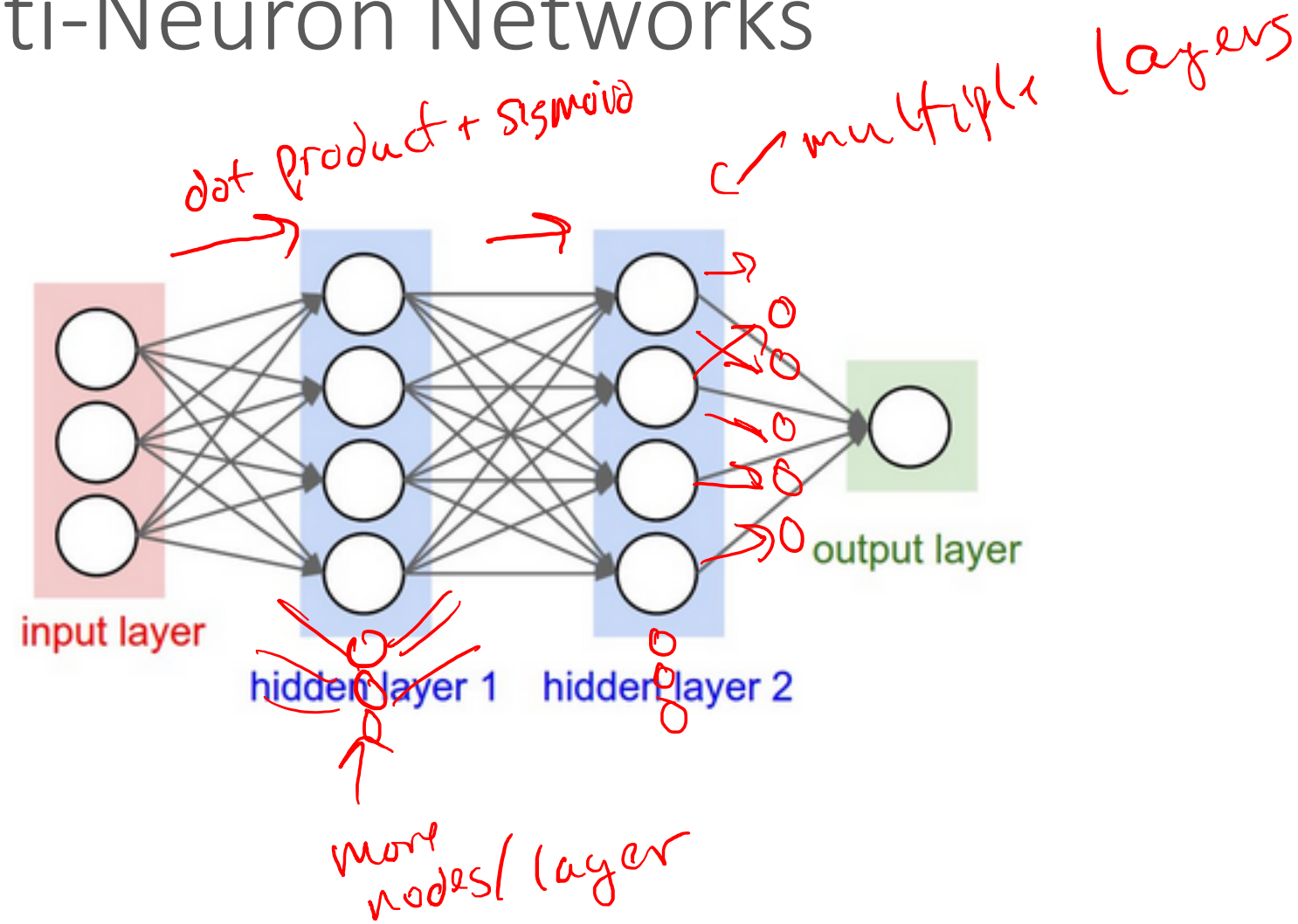
$$\overbrace{x_1 \cdot w_{31} + x_2 \cdot w_{32} + x_2 \cdot w_{33}}^{h_3}$$


neurons
neurons
h1
h2
h3
h4
Dog : 5
Cat : 100
input layer
hidden layer
output layer

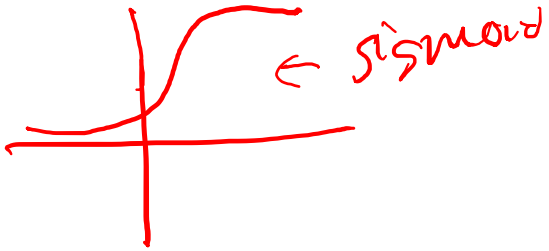
Max output decides



More Multi-Neuron Networks



More Vector Math

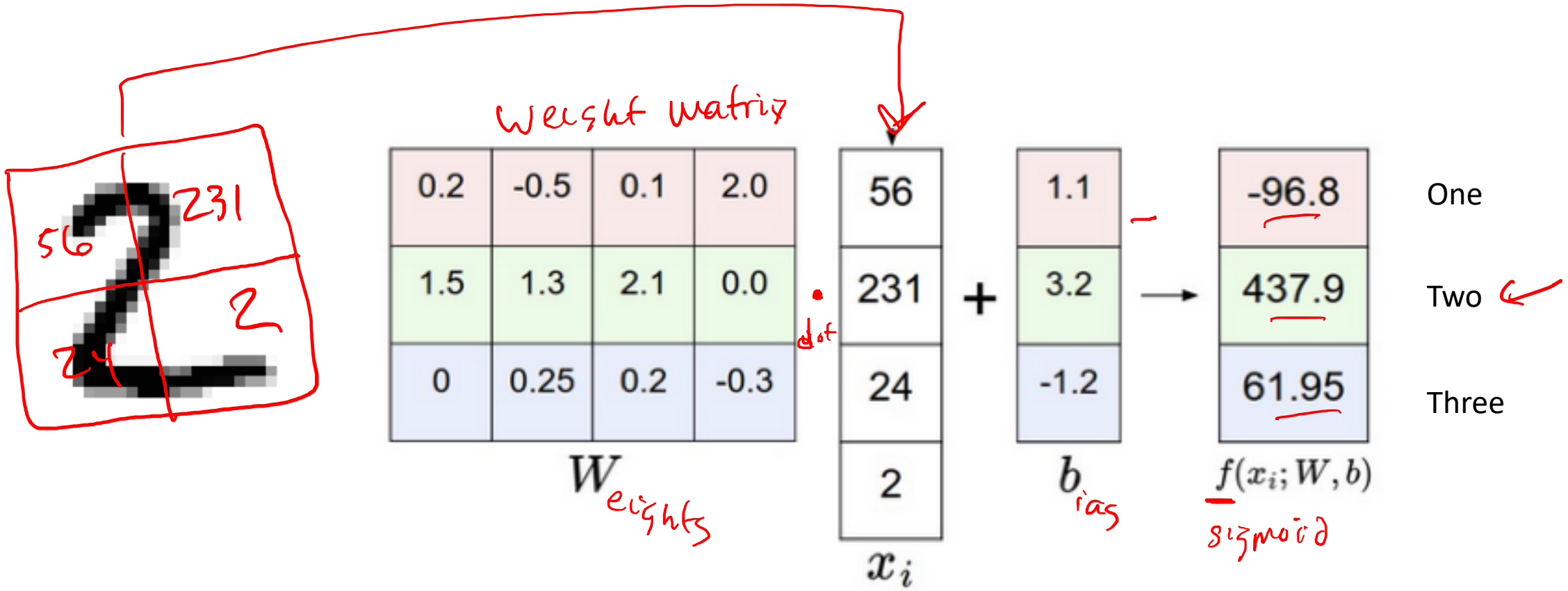



```
# forward-pass of a 3-layer neural network:
f = lambda x: 1.0/(1.0 + np.exp(-x)) # activation function (use sigmoid)
x = np.random.randn(3, 1) # random input vector of three numbers (3x1)
h1 = f(np.dot(W1, x) + b1) # calculate first hidden layer activations (4x1)
h2 = f(np.dot(W2, h1) + b2) # calculate second hidden layer activations (4x1)
out = np.dot(W3, h2) + b3 # output neuron (1x1)
```

h_1 outs

h_2 outs

does Not use sigmoid


weight matrix
56
231
24
2
0.2 -0.5 0.1 2.0
1.5 1.3 2.1 0.0
0 0.25 0.2 -0.3
dot
56
231
24
2
+
1.1
3.2
-1.2
-96.8
437.9
61.95
One
Two
Three
W eights
x_i
b ias
$f(x_i; W, b)$
sigmoid

Where do the Weights come from?

- PX Short Answer: From a file.
- PX Longer Answer:
  - Can re-“Train” yours for PX if you want.
  - Training steps in PX starter code.
- Real answer: Beyond scope of class. For more, take an actual Neural Networks class!

Next Time

- Look at Python code on Pynq
- Start to look at how to optimize it.

Vmware
ubuntu

16: Simple Neural Networks

Engr 315: Hardware / Software Codesign

Andrew Lukefahr
Indiana University